

```
git clone https://github.com/      /private-rag-agent
cd private-rag-agent
pip install -r requirements.txt
```

Private RAG Agent — 本地私有 AI 智能体

2026 AMD AI DevMaster 黑客松 · 赛道 2 (本地私有 AI Agent / Agentic AI 本地部署)

完全离线 · 本地推理 · 数据不出本机 · AMD Radeon GPU / ROCm 优化

□ 项目简介

一个全离线、本地私有化的多 Agent RAG 智能体：导入私有文档，通过"分解 → 并行研究 → 事实核查 → 综合"的协作管线回答复杂问题，全程不依赖任何云端 API。

核心卖点：

- 数据不出本机 —— 切片、向量化、检索、推理全部本地完成
- AMD GPU 本地推理 —— 多后端 (llama.cpp HIP / Ollama ROCm / vLLM) , 支持 Radeon Cloud
- 多 Agent 协作 —— Decomposer / 并行 Researchers / Fact-Checker / Synthesizer
- 可追溯 + 可核查 —— 混合检索 → cross-encoder 重排 → groundedness 事实核查 → 引用即点即查
- 多格式文档 —— PDF / Word / Markdown / Excel / PPT 全部支持

□ 快速开始

环境要求

- AMD Radeon GPU (推荐 RX 7900 XTX / W7900, 16GB+ 显存; 或 Radeon Cloud)
- ROCm 6.x (AMD Linux) / Ollama (Windows)
- Python 3.10+

1. 安装

```
# A Ollama
ollama pull qwen3:8b          # LLM
ollama pull bge-m3            # Embedding
ollama serve

# B llama.cpp + ROCm AMD GPU
# CMAKE_ARGS="-DGGML_HIP=ON" pip install llama-cpp-python
```

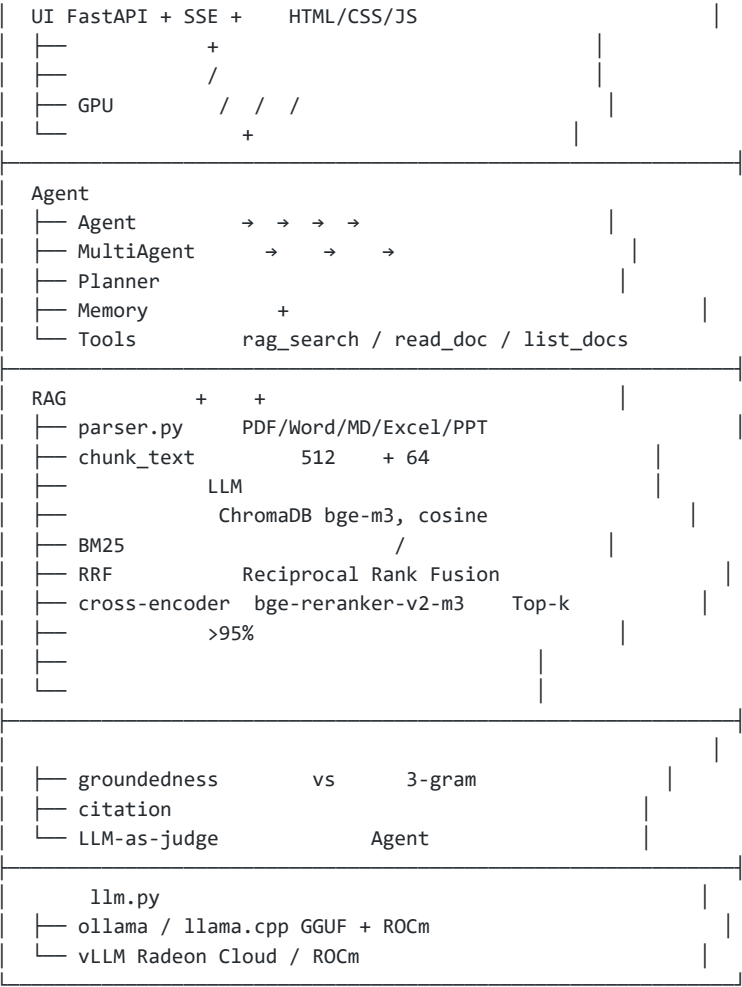
2. 启动本地推理服务

```
python main.py ingest data/docs/ #
python main.py ui                 # http://localhost:7860
```

3. 导入文档 + 启动网页

```
docker compose up -d --build
docker exec -it private-rag-ollama ollama pull qwen3:8b
docker exec -it private-rag-ollama ollama pull bge-m3
```

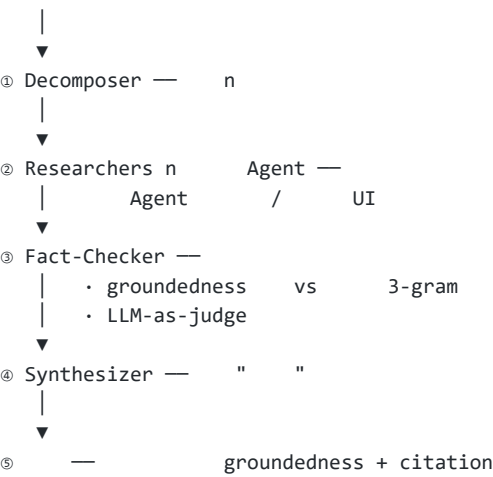
□ 一键容器化部署



在 AMD + ROCm 主机上取消 docker-compose.yml 中 ollama 的 devices 注释即可让 GPU 进容器。

□□ 架构

端到端链路



多 Agent 协作管线 ("严谨性"叙事核心)

```
rocminfo                # ROCm
rocm-smi                 # GPU / /
ollama ps                # 100% GPU
python benchmarks/bench_amd.py # tokens/s
```

并行动机（AMD 特色）：本地 GPU 上多个推理并发排队，串行 → 并行使多步研究任务的墙钟时间大幅下降；n 个子 Agent 共用一份 embedding / reranker 模型（模块级单例），显存占用只加一份。

□ AMD / ROCm 优化（赛道评分 40 分）

1. 多后端推理

方案	场景	说明	
llama.cpp (HIP)	单机单卡	CMAKE_ARGS="-DGGML_HIP=ON" ， GGUF 全 GPU offload	
Ollama (ROCm)	开箱即用	Linux 下自动 ROCm 后端， ollama ps 验证 100% GPU	
vLLM (ROCm)	高并发 / Radeon Cloud	Docker: rocm/vllm， 本项 目已内置对接	
量化	显存	速度	推荐
Q8_0	~8.2GB	快	追求精度
Q4_K_M	~4.6GB	最快	□ 均衡推荐
llama.cpp (HIP)	单机单卡	CMAKE_ARGS="-DGGML_HIP=ON" ， GGUF 全 GPU offload	
Ollama (ROCm)	开箱即用	Linux 下自动 ROCm 后端， ollama ps 验证 100% GPU	
vLLM (ROCm)	高并发 / Radeon Cloud	Docker: rocm/vllm， 本项 目已内置对接	
量化	显存	速度	推荐
Q8_0	~8.2GB	快	追求精度
Q4_K_M	~4.6GB	最快	□ 均衡推荐
llama.cpp (HIP)	单机单卡	CMAKE_ARGS="-DGGML_HIP=ON" ， GGUF 全 GPU offload	
Ollama (ROCm)	开箱即用	Linux 下自动 ROCm 后端， ollama ps 验证 100% GPU	
vLLM (ROCm)	高并发 / Radeon Cloud	Docker: rocm/vllm， 本项 目已内置对接	
量化	显存	速度	推荐
Q8_0	~8.2GB	快	追求精度

llama.cpp (HIP)	单机单卡	CMAKE_ARGS="-DGGML_HIP=ON" ， GGUF 全 GPU offload	
Q4_K_M	~4.6GB	最快	□ 均衡推荐
Ollama (ROCm)	开箱即用	Linux 下自动 ROCm 后端, ollama ps 验证 100% GPU	
vLLM (ROCm)	高并发 / Radeon Cloud	Docker: rocm/vllm, 本项 目已内置对接	
量化	显存	速度	推荐
Q8_0	~8.2GB	快	追求精度
Q4_K_M	~4.6GB	最快	□ 均衡推荐
vLLM (ROCm)	高并发 / Radeon Cloud	Docker: rocm/vllm, 本项 目已内置对接	
量化	显存	速度	推荐
Q8_0	~8.2GB	快	追求精度
Q4_K_M	~4.6GB	最快	□ 均衡推荐

2. 量化 (GGUF)

量化	显存	速度	推荐
Q8_0	~8.2GB	快	追求精度
Q4_K_M	~4.6GB	最快	□ 均衡推荐
Q8_0	~8.2GB	快	追求精度
Q4_K_M	~4.6GB	最快	□ 均衡推荐
Q8_0	~8.2GB	快	追求精度
Q4_K_M	~4.6GB	最快	□ 均衡推荐
Q4_K_M	~4.6GB	最快	□ 均衡推荐

3. GPU 资源节流 (小模型共享)

- embedding / reranker 用模块级单例，多 Agent 并行时全局只加载一份
- 检索重排按需懒加载，避免启动即占满显存

4. 验证与监控

```
private-rag-agent/  
├── agent/  
│   ├── agent.py          # Agent      / / /  
│   ├── multi_agent.py    # Agent      →  →  →  
│   ├── rag.py            # BM25 → RRF → rerank  
│   ├── verify.py         # groundedness + citation  
│   ├── llm.py            # ollama / llama.cpp / vLLM  
│   └── parser.py         # +
```

```
|   |— tools.py      #
|   |— memory.py   #
|   |— planner.py  #
|— ui/
|   |— server.py   # FastAPI + SSE
|   |— static/     # HTML/CSS/JS
|— benchmarks/    # AMD ROCm
|— config.yaml    # / /
|— main.py        #
|— deploy_rc.sh   # Radeon Cloud
|— requirements.txt
```

开发机估算 (NVIDIA RTX 4070 Laptop, Ollama qwen3:8b, CPU 推理) : 0.2~11 token/s
(视输出长度波动, 实测工件见 benchmarks/results/bench_local_20260806.txt)。切换到 Radeon
Cloud (RX 7900 / ROCm) 后, GGUF Q4_K_M + 全层 GPU offload 预计吞吐提升 5~10
倍, 正式提交的 benchmarks/ 会补上实测对比数据。

□ 目录结构

□ 提交信息 (Track 2)

- 赛道: Track 2 (私有 AI 智能体本地部署)
- 队伍: _____
- 演示视频: _____ (B站/油管链接)
- 提交截止: 2026-08-06 23:59 (北京时间)

□ 免责声明

本项目为黑客松参赛作品, 所有模型权重与文档数据均本地存储, 不收集任何用户数据。